

Big Data Lab

Lab 14: Project Integration Sprint & Final Preparation

Dr. Innocent Nyalala

IIT Madras Zanzibar
Department of Data Science and AI

Monday, 15 June 2026

DO.jpg

Today's Agenda

Review + Planning (2:00–2:45)

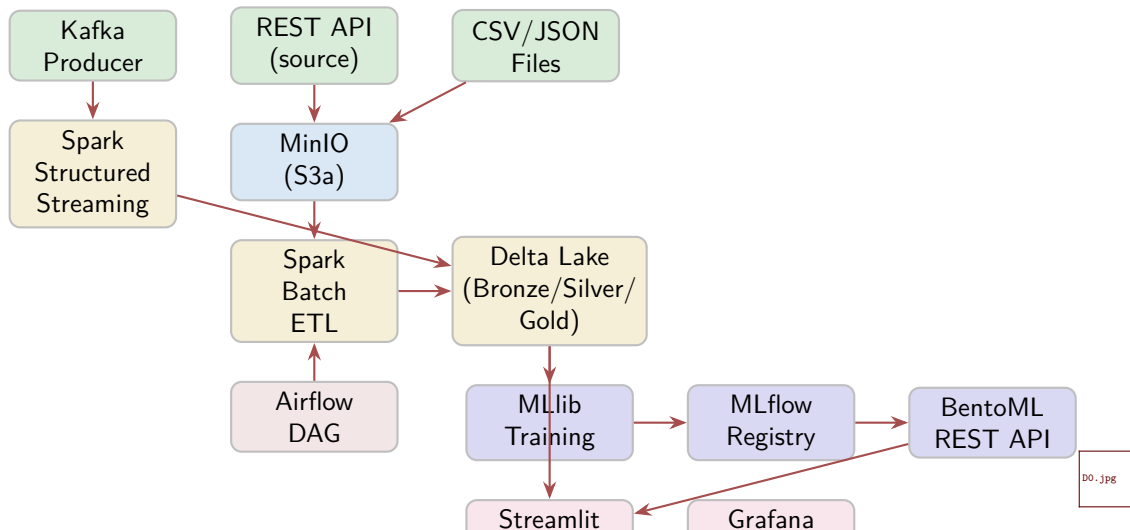
- Complete Big Data pipeline architecture recap
- Project integration checklist (layer by layer)
- Common integration bugs and fixes
- GitHub best practices for your repo
- Final report structure and requirements
- Presentation dos and don'ts

Supervised Build Time (2:45–4:30)

- Work on your project with instructor support
- Integration debugging clinic (one team at a time)
- End-to-end pipeline test: producer → Delta → model → API
- Dashboard and API live demo rehearsal
- Peer code review pairing
- Final questions before submission

0.jpg

Complete Project Reference Architecture



Integration Checklist: Layer by Layer

Must Have (required for pass)

- ☐ Kafka producer generates realistic data at rate ≥ 1 event/sec
- ☐ Data lands in Delta Lake (Bronze layer, at least 10K rows)
- ☐ Spark ETL cleans and transforms to Silver layer
- ☐ ML model trained on Silver data, logged to MLflow
- ☐ Model registered in MLflow as Production
- ☐ BentoML REST API serves predictions (tested with curl)
- ☐ Streamlit dashboard shows KPIs + 2 charts from Delta Lake
- ☐ Airflow DAG orchestrates at least 3 steps
- ☐ GitHub repository clean, README, notes

Bonus Features (+5 marks each, max +15)

- ☐ Great Expectations quality gate in Airflow DAG
- ☐ Drift detection with Evidently AI (automated alerts)
- ☐ Streaming dashboard (Streamlit + Kafka real-time)
- ☐ Docker Compose: entire project starts with 1 command
- ☐ A/B testing: 2 model versions in MLflow, compare live
- ☐ LLM chatbot interface over your data (natural language queries)

DO.jpg

GitHub Repository Structure

Required repo structure:

```
Z5008_Project_TeamN/  
  README.md #  
  project overview + setup guide  
    docker-compose.yml # (  
bonus) full stack  
  data/  
    raw/ #  
sample data (< 10MB)  
  schemas/ #  
column definitions  
  notebooks/  
    01_exploration.ipynb  
    02_feature_engineering.  
ipynb  
    03_model_training.ipynb  
    04_evaluation.ipynb  
  src/  
    producer.py #  
Kafka producer  
    pipeline.py #
```

README must include

1. Project title and team members
2. Problem statement (3–5 sentences)
3. Architecture diagram (image or ASCII)
4. Data description (source, size, schema)
5. Setup instructions:

```
git clone <repo>  
cd Z5008_Project_TeamN  
cp .env.example .env  
docker-compose up -d
```
6. How to run each component
(producer, pipeline, training, API, dashboard)
7. Results summary

DO.jpg

Final Report Structure (PDF, 8–12 pages)

Section	Content
Abstract	150-word summary of project approach, and results
1. Introduction	Problem, motivation, why Big tools are needed
2. Architecture	System diagram + justification each technology
3. Data Pipeline	Ingestion, transformation, Delta schema
4. ML Component	Model choice, training, evaluation metrics
5. Deployment	API design, latency benchmarks, test results
6. Monitoring	Drift detection setup, quality ga
7. Results	Key findings, limitations

Grading Rubric (50 marks)

Technical implementation (pipeline + ML + API)	20
Code quality and GitHub repo	5
Dashboard and visualisation	5
Final report quality	10
Live demo and presentation	10
Total	50

AI tools declaration: If you used GitHub Copilot, ChatGPT, or similar tools, list what was AI-assisted and

DO.jpg